디바이스트리란 무엇인가

보드를 설명하는 자료 구조 · QEMU virt의 DTB를 꺼내 읽고, 고치고, 부팅 단계마다 따라가며

PC는 장치를 스스로 알아낸다. PCI 장치는 버스를 훑으면 대답하고, 나머지는 펌웨어가 ACPI 표로 알려 준다. 임베디드 SoC의 장치 대부분은 그렇지 않다. 직렬 포트는 `0x10000000`에 있다고 누가 말해 주지 않으면 커널은 알 길이 없다. **디바이스트리**는 "이 보드에는 무엇이 어디에 있는가"를 적은 자료 구조다. 커널 코드에서 보드 정보를 떼어 내어, 커널 바이너리 하나가 여러 보드에서 돌게 한다. 이 문서는 QEMU `virt` 보드의 디바이스트리를 꺼내 읽고, 바이너리를 뜯어 보고, 부팅 단계마다 누가 무엇을 고치는지, 커널이 그것으로 드라이버를 어떻게 붙이는지 확인한 기록이다.

2026년 10월 21일 기록. 대상: QEMU 11.1.2 (`-machine virt`), dtc 1.6.1, OpenSBI v1.8.1, U-Boot v2024.10, Linux v6.12.59, xv6-riscv `06aad25`. 부팅 환경은 함께 쓴 ``부트로더란 무엇인가''와 같다. 화면 그림은 tmux에서 실제로 실행한 뒤 받은 글자를 창 모양으로 그린 것이고, 건너뛴 줄은 `...`로 표시했다. 밑줄 친 소스 위치를 누르면 GitHub의 그 줄이 열린다(태그 고정).

1. 한눈에 보기

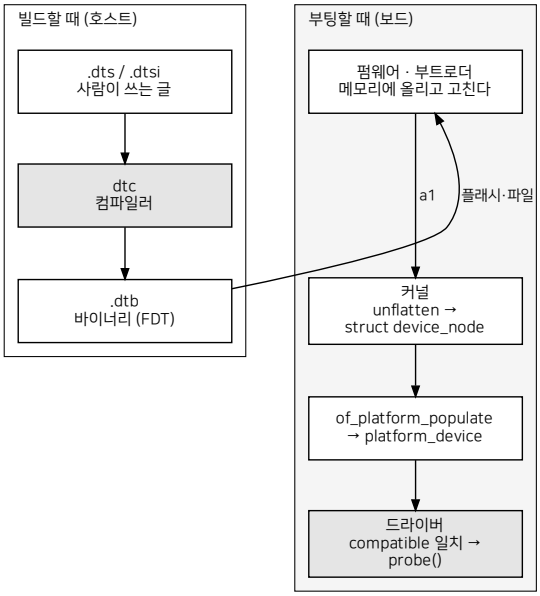


그림 1. 디바이스트리의 일생. 사람이 쓴 `.dts`가 컴파일되어 `.dtb`가 되고, 부트로더를 거쳐 커널에 들어가 드라이버의 `probe()`까지 간다.

용어	뜻
DTS (`.dts`)	Device Tree Source. 사람이 쓰는 글. 보드마다 하나
DTSI (`.dtsi`)	여러 보드가 함께 쓰는 부분(보통 SoC). `.dts`가 `#include`한다
DTB (`.dtb`)	Device Tree Blob. `dtc`가 만든 바이너리. 부트로더가 커널에 넘기는 것
FDT	Flattened Device Tree. DTB의 형식 이름. 트리를 바이트열 하나로 펼쳤다는 뜻
dtc	Device Tree Compiler. DTS ↔ DTB를 양쪽으로 바꾼다

2. 꺼내 보기

QEMU는 `virt` 보드를 만들 때 그 보드의 디바이스트리도 함께 만든다. `dumpdtb`로 파일에 받아 `dtc`로 글로 되돌렸다.

```
$ qemu-system-riscv64 -machine virt,dumpdtb=virt.dtb -m 128M -smp 3 -nographic
$ ls -l virt.dtb
-rw-rw-r-- 1 user user 7108 Oct 21 00:37 virt.dtb
$ dtc -I dtb -O dts -o virt.dts virt.dtb
$ wc -l virt.dts
267 virt.dts
```

267줄 가운데 뼈대만 그리면 다음과 같다.

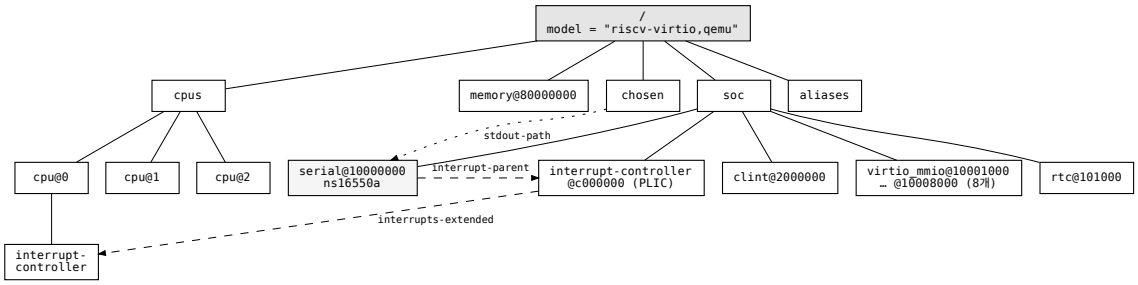


그림 2. QEMU virt 디바이스트리의 뼈대(일부). 실선은 부모-자식, 점선은 phandle로 다른 노드를 가리키는 참조다.

직렬 포트 노드와 그 주변을 그대로 옮기면 이렇다.

```
/ {
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    compatible = "riscv-virtio";
    model = "riscv-virtio,qemu";

    memory@80000000 {
        device_type = "memory";
        reg = <0x00 0x80000000 0x00 0x8000000>;
    };
    chosen {
        stdout-path = "/soc/serial@10000000";
    };
    soc {
        #address-cells = <0x02>;
        #size-cells = <0x02>;
        compatible = "simple-bus";
        ranges;

        serial@10000000 {
```

```
        interrupts = <0x0a>;
        interrupt-parent = <0x07>;
        clock-frequency = "\08@";
        reg = <0x00 0x10000000 0x00 0x100>;
        compatible = "ns16550a";
    };
    interrupt-controller@c000000 {
        phandle = <0x07>;
        compatible = "sifive,plic-1.0.0\0riscv,plic0";
        ...
```

3. 문법: 노드와 속성

디바이스트리는 **노드**의 트리이고, 노드마다 **속성**(이름 = 값)이 붙는다. 노드 이름은 "이름@단위주소다. `serial@10000000`의 `10000000`은 그 장치의 첫 주소로, 같은 이름의 노드를 구별하려고 붙인다.

속성	값의 형태	뜻
`compatible`	문자열 목록	"나는 이것과 호환된다". 구체적인 것부터 쓴다. 드라이버를 고르는 열쇠(7절)
`reg`	32비트 셀 목록	주소와 크기의 쌍. 셀 개수는 **부모**의 `#address-cells`, `#size-cells`가 정한다
`interrupts`	셀 목록	인터럽트 번호. 어느 컨트롤러의 번호인지는 `interrupt-parent`가 정한다
`interrupt-parent`	phandle	다른 노드를 가리키는 32비트 번호. 가리켜지는 쪽에 `phandle = <0x07>`이 있다
`status`	문자열	`"okay"`(또는 없음)면 쓰고, `"disabled"`면 없는 셈 친다(8절)
`model`	문자열	보드 이름. 사람이 읽으라고 둔다

3.1 reg 읽는 법

`soc` 노드가 `#address-cells = <2>`, `#size-cells = <2>`이므로, 자식의 `reg`는 셀 4개가 한 쌍이다. 64비트 주소와 64비트 크기를 각각 32비트 셀 두 개로 쓴 것이다.

	주소 상위	주소 하위	크기 상위	크기 하위
`serial@10000000`	`0x00`	`0x10000000`	`0x00`	`0x100`
	→ 주소 `0x10000000`		→ 크기 256 B	
`memory@80000000`	`0x00`	`0x80000000`	`0x00`	`0x8000000`
	→ 주소 `0x80000000`		→ 크기 128 MB	

3.2 phandle: 트리 밖으로 가는 화살표

트리 구조는 "무엇이 무엇 안에 있나"만 말한다. 직렬 포트의 인터럽트가 PLIC로 간다는 관계는 트리로 그릴 수 없어서 번호로 잇는다. PLIC 노드에 `phandle = <0x07>`이 있고, 직렬 포트·RTC·virtio 등 11개 노드가 `interrupt-parent = <0x07>`로 그것을 가리킨다. 직렬 포트의 인터럽트는 "PLIC의 10번"이다. xv6의 `UART0_IRQ 10`(`memlayout.h:22`)과 같은 값이다.

핵심 디바이스트리는 노드의 트리다. 장치마다 `compatible`(무엇인가), `reg`(어디 있나), `interrupts`(무

엇으로 부르나)를 적는다.

4. 바이너리: DTB 안쪽

오프셋	블록	내용
0x0000	헤더 (40 B)	magic 0xd00dfeed, 전체 크기, 각 블록의 위치, 버전 17
0x0028	예약 메모리 맵	커널이 건드리면 안 되는 구간 목록 (여기선 비어 있음)
0x0038	구조 블록 (6552 B)	BEGIN_NODE "이름" · PROP (길이, 이름 오프셋, 값) · END_NODE ⋯ END
0x19d0	문자열 블록 (500 B)	속성 이름을 한 번씩만: "model\0compatible\0…"
0x1bc4	끝 (totalsize = 7108)	

그림 3. DTB의 배치. 수치는 virt.dtb(7108 바이트)에서 읽었다.

```
$ xxd virt.dtb | head -3
00000000: d00d feed 0000 1bc4 0000 0038 0000 19d0   magic, totalsize, off_struct, off_strings
00000010: 0000 0028 0000 0011 0000 0010 0000 0000   off_rsvmap, version 17, last_comp 16
00000020: 0000 01f4 0000 1998 ...                   size_strings, size_struct
```

모든 정수는 빅엔디언이다. 구조 블록은 토큰의 나열이다. 1 = 노드 시작, 2 = 노드 끝, 3 = 속성, 9 = 끝. 구조 블록의 처음을 풀어 보면 이렇다.

```
$ xxd -s 0x38 -l 32 virt.dtb
00000038: 0000 0001 0000 0000 0000 0003 0000 0004
00000048: 0000 001d 0000 0002 ...
   00000001             BEGIN_NODE
   00000000             name "" (root), padded to 4 bytes
   00000003             PROP
   00000004             len = 4
   0000001d             nameoff = 0x1d -> strings[0x1d] = "#address-cells"
   00000002             value = 2
```

속성 이름은 구조 블록에 직접 쓰지 않고, 문자열 블록의 오프셋만 쓴다. reg나 compatible처럼 수십 번 나오는 이름을 한 번만 저장하려는 것이다.

```
$ xxd -s 0x19d0 -l 48 virt.dtb
000019d0: 6d6f 6465 6c00 636f 6d70 6174 6962 6c65  model.compatible
000019e0: 0023 7369 7a65 2d63 656c 6c73 0023 6164  .#size-cells.#ad
000019f0: 6472 6573 732d 6365 6c6c 7300 7261 6e67  dress-cells.rang
```

0x19d0 + 0x1d = 0x19ed에 #address-cells가 있다. 부트로더 문서 실험 2에서 gdb로 본 a1의 첫 4바이트 d0 0d fe ed가 바로 이 파일의 첫 4바이트다.

5. 보드마다 다르다: 메모리만 바꿔 보기

같은 virt 보드에 RAM만 128 MB에서 256 MB로 바꿔 DTB를 다시 뽑았다.

```
$ qemu-system-riscv64 -machine virt,dumpdtb=virt256.dtb -m 256M -smp 3 -nographic
$ diff <(dtc -I dtb -O dts virt.dtb) <(dtc -I dtb -O dts virt256.dtb)
```

```
33c33
<          reg = <0x00 0x80000000 0x00 0x8000000>;
---
>          reg = <0x00 0x80000000 0x00 0x10000000>;
```

(`/chosen/rng-seed`는 실행할 때마다 바뀌는 난수라 빼고 비교했다.) 메모리 노드 하나만 달라졌다. Linux는 부팅 때 이 값을 읽으므로 커널을 다시 빌드하지 않아도 256 MB를 다 쓴다. xv6는 다르다. 메모리 끝을 `PHYSTOP = KERNBASE + 128 MB`로 소스에 박아 두어 (`memlayout.h:44`), RAM을 늘려도 128 MB만 쓴다.

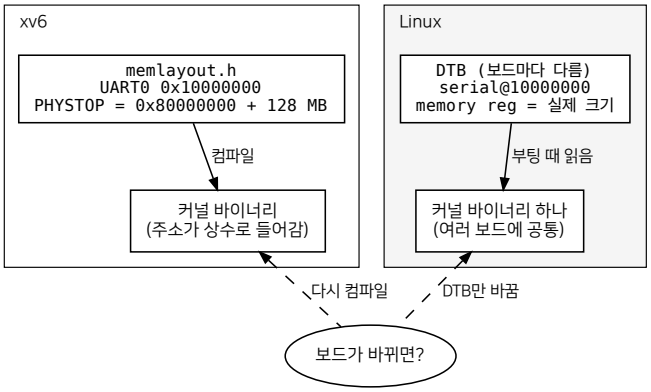


그림 4. 하드웨어 주소를 아는 두 방법.

6. 부팅 중 여러 손을 거친다

DTB는 만들어진 그대로 커널에 가지 않는다. 단계마다 자기가 아는 것을 덧붙인다.

단계	크기	한 일 (이번 실험에서 확인한 것)
QEMU	7108 B	보드 구성대로 생성 (`dumpdtb`)
OpenSBI	11351 B	`/reserved-memory`에 자기 영역 두 개(`mmode_resv0`, `mmode_resv1`)를 더함
U-Boot	23638 B	사본을 만들고, `booti` 직전 `/chosen`에 `bootargs`와 initramfs 위치를 씀. 고칠 자리를 위해 여유 공간을 붙인 크기다(`booti` 출력 `0x86ee5170-0x86eeadc6`)
Linux	–	읽기만 한다. `/proc/device-tree`로 사용자에게 보여 준다

U-Boot · OpenSBI가 더한 노드

```
=> fdt list /reserved-memory
reserved-memory {
        #address-cells = <0x00000002>;
        #size-cells = <0x00000002>;
        ranges;
        mmode_resv1@80000000 {
        };
        mmode_resv0@80040000 {
        };
};
```

그림 5. U-Boot에서 본 `/reserved-memory`. QEMU가 만든 원본에는 없던 노드다.

U-Boot · booti 전의 /chosen

```
=> fdt print /chosen
chosen {
        stdout-path = "/soc/serial@10000000";
        rng-seed = <...>;
};
```

그림 6. `booti` 전의 `/chosen`. `bootargs`가 없다.

커널이 뜬 뒤 `/init`이 `/proc/device-tree/chosen/bootargs`를 읽으면 `console=ttyS0 earlycon`이 나온다(그림 8). U-Boot가 `fdt_chosen`(`boot/image-fdt.c:598`)으로 써 넣은 것이다. 커널 로그의 `OF: reserved mem: ... mmode_resv1@80000000`은 OpenSBI가 써 둔 노드를 커널이 읽은 결과다.

핵심 디바이스트리는 단계 사이의 전달 문서다. 펌웨어는 "내 자리는 피하라"를, 부트로더는 "커널 인자와 initramfs는 여기"를 적어 넘긴다.

7. 커널이 쓰는 법

7.1 두 번 읽는다

1. **일찍, 펼친 채로**: 메모리 관리자가 서기도 전에 `parse_dtb()`(`arch/riscv/kernel/setup.c:261`)가 `early_init_dt_scan()`(`drivers/of/fdt.c:1175`)으로 바이너리를 직접 훑는다. `model`, `/memory`, `/reserved-memory`, `/chosen`만 본다. 메모리가 얼마인지 알아야 메모리 관리자를 세울 수 있기 때문이다.
2. **나중에, 트리로**: 메모리 관리자가 선 뒤 `unflatten_device_tree()`(`fdt.c:1210`)가 DTB 전체를 `struct device_node`의 트리로 풀어 놓는다. 이후 드라이버들은 `of_*` 함수로 이 트리를 묻는다.

7.2 노드가 장치가 되고, 드라이버가 붙는다

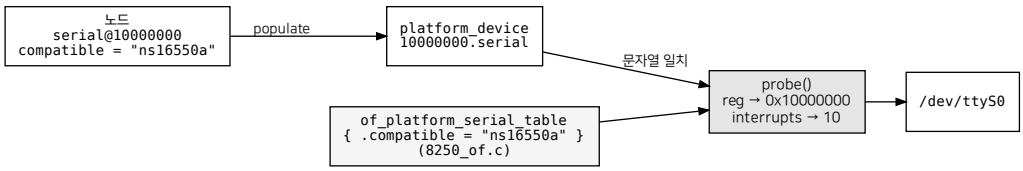


그림 7. 노드 `serial@10000000`이 `ttyS0`이 되기까지.

`of_platform_default_populate_init()`(`drivers/of/platform.c:503`)이 루트의 자식과 `compatible = "simple-bus"`인 노드(여기선 `soc`)의 자식을 `platform_device`로 만든다. 장치 이름은 "단위 주소.노드 이름"이라 `10000000.serial`이 된다. 그다음 드라이버 쪽에서 `of_match_table`의 `compatible` 문자열과 노드의 `compatible`을 비교한다. 직렬 포트 드라이버의 표에 `"ns16550a"`가 있으므로 (`drivers/tty/serial/8250/8250_of.c:322`) `of_serial` 드라이버가 붙고, `probe()`가 `reg`에서 주소를, `interrupts`에서 IRQ를 읽어 `ttyS0`을 만든다.

```
[    0.878892] 10000000.serial: ttyS0 at MMIO 0x10000000 (irq = 12, ...) is a 16550A
```

`irq = 12`는 디바이스트리의 10이 아니다. 10은 PLIC 안에서의 번호(hwirq)이고, 12는 커널이 전역으로 다시 매긴 번호(virq)다.

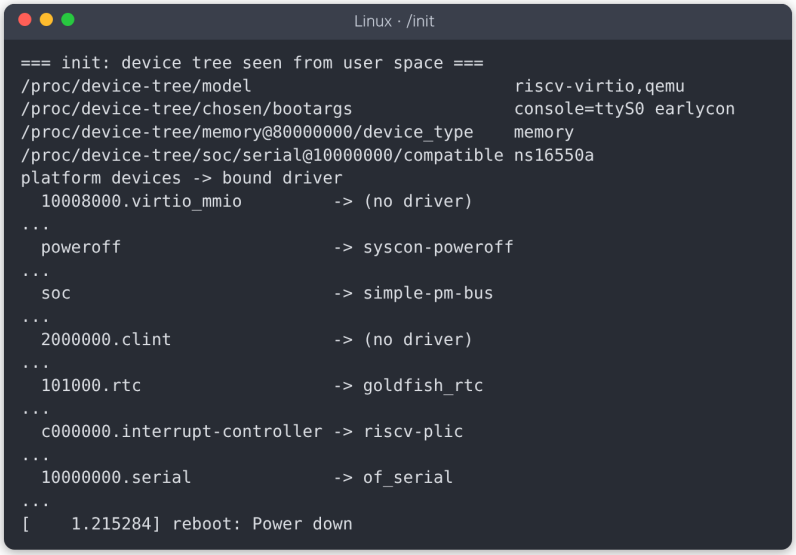


그림 8. `/init`이 사용자 공간에서 읽은 디바이스트리와, 장치마다 붙은 드라이버(일부).

`(no driver)`인 `virtio_mmio` 여덟 칸은 노드는 있지만 QEMU에 virtio 장치를 하나도 달지 않은 빈 칸이다. 드라이버가 장치 ID 0을 읽고 `-ENODEV`로 물러난다(`drivers/virtio/virtio_mmio.c:656--662`). `clint`는 S-mode 커널이 직접 쓰지 않는 장치(OpenSBI가 M-mode에서 쓴다)라 붙는 드라이버가 없다.

핵심 노드는 `platform_device`가 되고, 드라이버는 `compatible` 문자열로 자기 장치를 찾는다. 주소와 IRQ는 드라이버 코드가 아니라 디바이스트리에서 온다.

8. 실험: 디바이스트리를 고쳐 부팅하기

원본 `virt.dts`를 두 군데 고쳐 `my.dtb`를 만들고, QEMU의 `-dtb`로 이 파일을 대신 넘겼다.

```
$ diff virt.dts my.dts
7c7
<       model = "riscv-virtio,qemu";
---
>       model = "my-board,experiment";
158a159
>                       status = "disabled";      # in rtc@101000
$ dtc -I dts -O dtb -o my.dtb my.dts
```

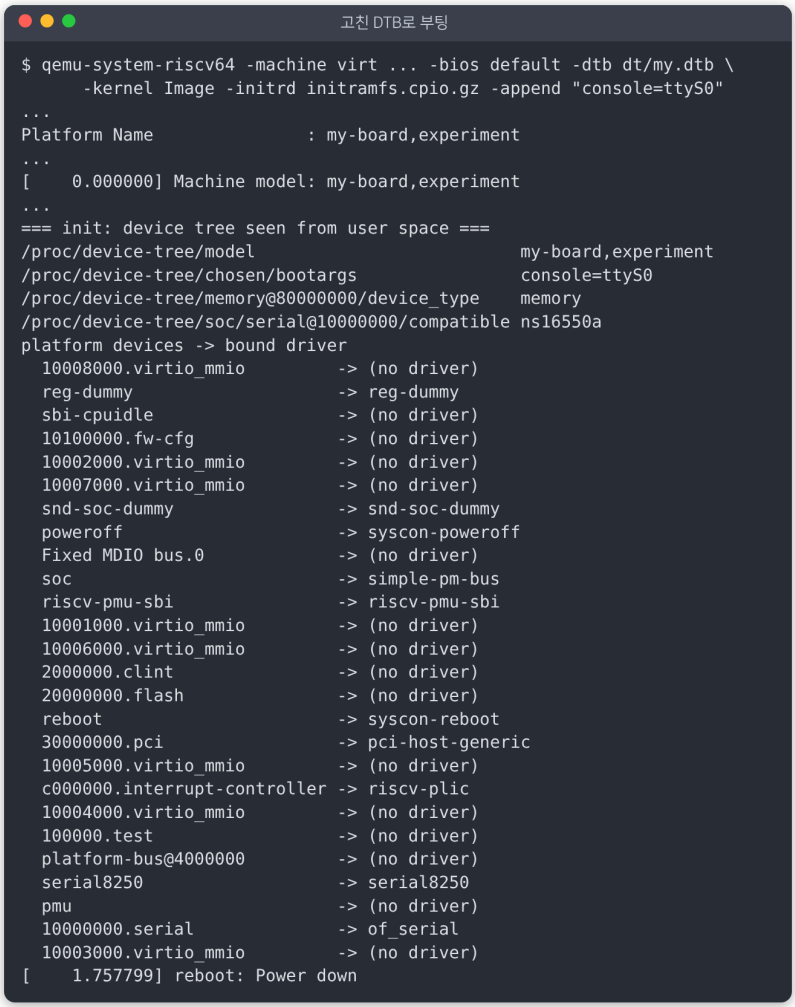


그림 9. 고친 DTB로 부팅. 이름이 OpenSBI, 커널, 사용자 공간까지 그대로 전달됐고, 장치 목록에서 `101000.rtc`가 사라졌다.

- `model`: OpenSBI의 `Platform Name`, 커널의 `Machine model`, `/proc/device-tree/model`이 모두 `my-board,experiment`로 바뀌었다. 셋 다 같은 문자열을 읽고 있었다는 증거다.
- `status = "disabled"`: 원래 부팅에서는 `goldfish_rtc 101000.rtc: registered as rtc0`가 찍히고 목록에 `101000.rtc -> goldfish_rtc`가 있었다. 고친 뒤에는 둘 다 없다. 장치를 만들기 전에 `of_device_is_available()`이 `status`를 보고 건너뛴다(`drivers/of/platform.c:171`). 하드웨어(QEMU의 RTC)는 그대로 있지만 커널에게는 없는 장치다.

핵심 커널을 다시 빌드하지 않고 DTB만 바꿔 보드의 모습을 바꿨다. 디바이스트리가 하드웨어 설명을 코드에서 떼어 냈다는 말의 뜻이다.

9. 실제 보드에서 더해지는 것

- **SoC .dtsi + 보드 .dts**: 칩 제조사가 SoC 전체(CPU, 버스, 컨트롤러 주소)를 `.dtsi`로 주고, 보드 제조사는 그것을 `#include`한 뒤 실제로 배선한 장치만 `status = "okay"`로 켜고 센서, PHY, 핀 설정을 더한다.
- **QEMU처럼 만들어 주지 않는다**: 실제 보드의 DTB는 커널 소스(`arch/riscv/boot/dts/`)나 벤더 BSP에서 빌드해 플래시에 넣고, 부트로더가 커널과 함께 읽어 넘긴다. FIT 이미지에 커널과 DTB를 묶는 경우가 많다.

- **clock, pinctrl, regulator, gpio**: QEMU virt에는 거의 없지만 실제 SoC에서는 장치마다 클록, 핀 기능, 전원, 리셋 선을 phandle로 이어 적는다. 디바이스트리의 대부분이 이것이다.
- **오버레이(`.dtbo`)**: 기본 DTB 위에 일부 노드만 덮어쓰는 조각. 확장 보드나 카메라 모듈을 바꿀 때 쓴다.
- **바인딩 문서**: 노드마다 어떤 속성이 필요한지는 커널의 `Documentation/devicetree/bindings/`에 YAML로 정해져 있고, `make dtbs_check`로 검사한다.

10. 정리

1. 디바이스트리는 스스로 알려 주지 않는 하드웨어(주소, 인터럽트, 메모리 크기)를 적은 자료 구조다. 커널 코드에서 보드 정보를 떼어 낸다.
2. 사람은 `.dts`를 쓰고, `dtc`가 `.dtb`(FDT)로 바꾸며, 부트로더가 그 주소를 `a1`에 넣어 커널에 넘긴다.
3. 노드는 "이름@주소, 핵심 속성은 `compatible`, `reg`, `interrupts`다. `reg`의 셀 수는 부모의 `#address-cells`, `#size-cells`가 정하고, 다른 노드는 phandle로 가리킨다.
4. DTB는 헤더, 예약 메모리 맵, 구조 블록(토큰), 문자열 블록으로 되어 있고 모두 빅엔디언이다. 매직은 `0xd00dfeed`다.
5. 부팅 중 OpenSBI는 `/reserved-memory`를, U-Boot는 `/chosen`의 `bootargs`와 initramfs 위치를 덧붙인다(7108 B → 11351 B → 23638 B).
6. 커널은 일찍 펼친 채로 메모리·`/chosen`을 읽고, 나중에 트리로 풀어 노드를 `platform_device`로 만든 뒤 `compatible`로 드라이버를 붙인다.
7. DTB만 고쳐도(`model`, `status = "disabled"`) 커널을 다시 빌드하지 않고 커널이 보는 보드가 바뀐다. xv6는 주소를 소스에 박아 두어 그렇게 할 수 없다.